

Requirements Document.

Background: The system will identify attackers using tools that trap the attacker into receiving false information, and through these actions we will monitor these events in a separate admin page with secured access.

1. High Level

The system is divided into two separate sides:

- A. The Hidden Side (Admin Dashboard): Intended for security administrators for monitoring attackers, configuration, and receiving alerts.
- B. The Visible Side (The Decoy Layer) A collection of pages, API and fake services designed to lure the attacker, identify their intentions, and waste their time while they search for their target.

2. Management and Monitoring Module (Admin Dashboard)

The control center must be secured at the highest level and hidden from the external network.

A. Login Screen (Real Admin Login)

Mechanism: Two-factor authentication 2fa, every defined admin will have a complex password, a two-factor authentication mechanism via a free app (e.g., netiq auth) in order to verify authorized access and secure the login page in the most efficient manner.

Protection: Access to this screen will only be allowed from an authorized IP address; an access attempt from an unknown IP will immediately trigger a trap (sometimes showing a 404 page, or redirecting to a fake login trap)

B. Home Page (Dashboard Overview)

Dashboard: A visual display showing the real-time threat status

- Attack Map: Visualization of the ip addresses of attackers on the map.
- Wasted Resources Counter: A metric showing how much time and resources the system has “burned” on attackers (e.g., “We delayed attackers by 4 cumulative hours today”)
- Live Alerts: A running list (logs) of recent actions. (e.g., sql injection attempt detected on login)

C. Attacker Profile Page (attacker fingerprint)

Clicking on a specific attacker in the dashboard will open a page with all collected information.

- network info: IP address, Internet Service Provider (isp), Geographic location
- device fingerprint: Browser type, operating system, screen resolution, device type, etc.
- attack pattern: Which traps did the attacker fall into? (Tried sql? Backup file search?)
- risk score: Automatic score for the attacker's danger level

3. **The Traps Module (The Decoy Layer) - Details by Attack Type**

Here we define how the system responds to each attack. The goal is not to block immediately, but to “feed” the attacker false information so they continue wasting time.

A. SQL Injection

The attacker attempts to inject code into input fields (e.g., ' OR 1=1--).

- **The Trap**: A search or login form that appears vulnerable
- **The Response**: Instead of returning a real error or blocking, the system will return a fake Database Error that looks convincing. (e.g., MySQL Error: Syntax check failed near line 1).
- **Exhaustion**: If the attacker persists, the system will start responding more slowly and return fictitious tables with fabricated usernames and passwords. The attacker will waste hours trying to crack passwords that do not exist.

B. Brute Force / Credential Stuffing

The attacker attempts to guess passwords in a login form.

- **The Trap**: A login page for an Admin Panel that looks prestigious and sensitive.
- **Mechanism**: Every fifth attempt fails, on the tenth attempt the system lets them in even if the password is wrong.
- **Inside the Trap**: The attacker gains access to a fake management interface (Honeypot Dashboard) - every button they click does nothing except send alerts to the real admin and document their behavior.

C. Sensitive File Scanning

The attacker attempts to access files such as .env, config.json, backup.sql.

The Trap: The system will not return a 404; instead it will serve a fake env. file. The content (Honeytoken) inside the file will contain API keys and passwords for sensitive resources

Monitoring: These keys are “marked.” The moment someone tries to use one of them elsewhere, we know with certainty that this is the attacker who stole the file

D. XSS - Cross Site Scripting

The attacker attempts to inject scripts (<script>alert(1)</script>).

- **The Response:** The system will accept the script and display it back to the attacker, but within an isolated environment (sandbox) that does not harm the server.
- **The Deception:** The system will display an “admin session captured” message to make them think they succeeded and wait for something to happen, while we collect information on them.

4. **Exhaustion and Delay Mechanisms (Resource Exhaustion logic)**

Goal: To cause the attacker to burn CPU, memory, and valuable time.

A. HTTP Trait

When the system definitively identifies an attacker, their subsequent requests (request) will be answered very slowly; the server will send one byte of data every 10 seconds, which keeps the attacker's connection open and prevents them from quickly scanning other pages.

B. Infinite Redirects:

Certain pages will lead to infinite redirect loops or create an endless directory tree e.g., (/admin/v1/backup/2023/jan/files/...) in order to drive automatic scanners (Scrapers/Bots) crazy.

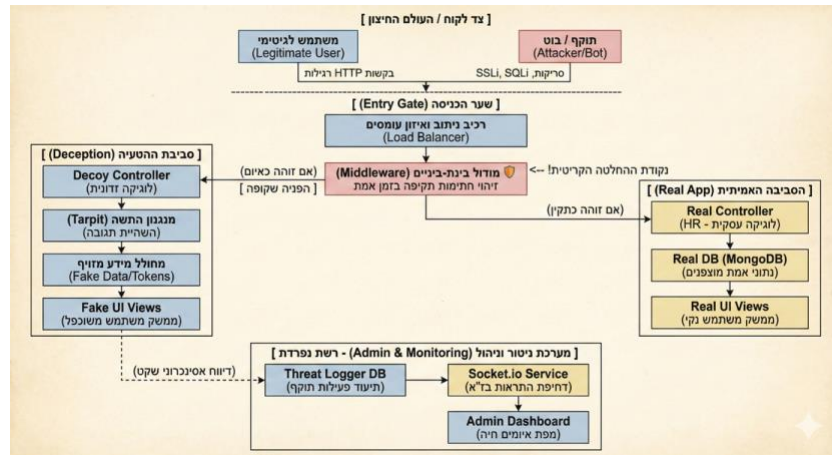
C. Data Bomb

If the attacker tries to download a “database backup,” we send a file composed of zeros with a size of 100GB that will saturate their bandwidth and fill their hard drive.

5. **Technical Architecture**

In order to implement this in node.js, the structure should be as follows:

- Central middleware: Every request passes through a “filter” — if the ip is flagged as malicious —> redirect to the deception. If legitimate —> continue normally.
- logger: Using a database (mongodb) for fast logging of every attacker action.
- socket.io: For real-time data transfer to the admin page (in order to see everything live)



Attacker Identification Process – Example:

Step A: middleware Is the IP known as an attacker?

Does the request contain suspicious characters?

Step B: The Decision Moment — Executing the Redirect:

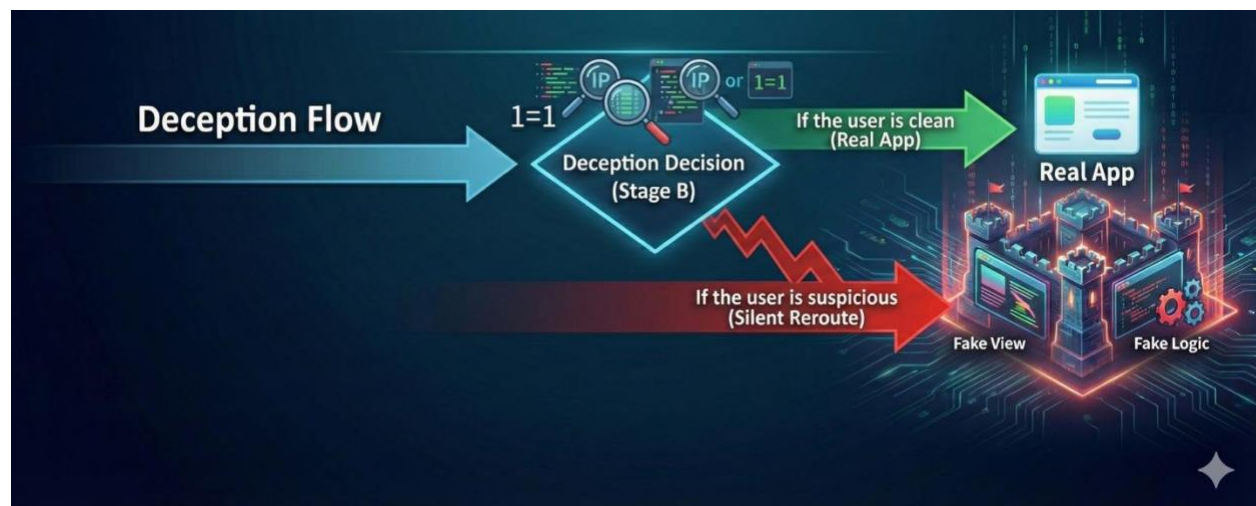
If the user is clean: The system will allow them to proceed to the regular application.

If the user is suspicious: The system will perform a silent reroute to “deception module”

This is a separate collection of files, Controllers and Views in the code, which is essentially a “shadow website.” This module contains:

Fake View: HTML pages that look exactly like the real pages but whose underlying code is completely different

Fake Logic: Functions that return fabricated errors, false data, or perform intentional delays



User Interface Specification – The Legitimate Side (The Real App)

Design Concept

Style: Corporate

Goal: To convey credibility. It should look like an internal system of a real company

Visual Technology: Use of a well-known design library

Screen Details (Flow)

Screen 1: Public Landing Page (Public Landing Page)

This is the page that everyone sees when they enter the main address.

- **Visuals:**
 - Logo of the fictional company in the top right corner.
 - Large image (Hero Image) of “smiling people in the office.”
 - Marketing text: “Welcome to InnoTech internal services”.
- **Functionality:**
 - Prominent button: “**Employee Login**” (leads to the login screen).
 - Secondary button: “Contact IT Support” (a static form that does nothing, only adds credibility).
- **Attack Context:** This is the first place the attacker's scanners (Scanners) will check. It is completely clean of traps so as not to raise suspicion.

Screen 2: The Login Page (The Gateway) – The Critical Point

This is the most important page. It appears *absolutely identical* for a regular user and for an attacker, but its behavior changes after the click.

- **Visuals:**
 - A box in the center of the screen.
 - Fields: Username, Password.
 - Link “Forgot Password?”.
- **The Legitimate Scenario**
 - A user enters the correct username/password → redirected to **Screen 3 (Dashboard)**.
 - Loading time: Immediate (0.5 seconds)
- **The Malicious Scenario**
 - An attacker types ' OR '1'='1.
 - **Visually:** They see (Loading...) spinning for a bit too long (artificially)
 - **Result:** They are redirected to the “Error Page of the Deception Layer without knowing they have left the real system.

Screen 3: Legitimate Home Page (Real User Dashboard)

This is what an employee sees when they log in correctly.

- **Visuals:**
 - Top Bar (Navbar): "Hello, User Name", Logout button.
 - Sidebar (Sidebar): My Profile, Salary Slips, Documents, Settings.
 - Screen Center (Widgets):
 - "Company Announcements"
 - "Remaining Vacation Days"
 - "Open Tasks."
 -
- **Importance for the Specification:** The real system must respond quickly and work smoothly. This is our Baseline for comparison against the slowness of the deception system.

Screen 4: Personal Profile Page (My Profile - Read Only)

- **Visuals:** A form displaying: name, email, phone, address.
- **Functionality:** An "Edit" button that opens a pop-up for editing.
- **Attack Context:**
 - **In the Real System:** Data is pulled from a real database (MongoDB/SQL).
 - **In the Deception System:** If an attacker managed to reach here (in the fake dashboard), they will see fabricated data (John Doe, address: 123 Fake St), and the fields will contain XSS traps (if they try to change their name to script code, the system will "swallow" it and log an alert

Screen 5: Document Search Area (Internal Search)

- **Visuals:** A large search bar "Search Employee Database".
- **Legitimate Functionality:** Searching for an employee name returns real results.
- **Attack Context:** This is a classic area for XSS and SQL Injection.
 - A regular user searches for "Cohen" → receives results.
 - An attacker searches for <script>... → nothing happens on the page, but behind the scenes they are flagged.

Information Flow Summary

Entry: All visitors arrive at the same Landing Page.

1. **Filtering (Login):**
 - Authorized user (User) → **Path A** (Real App Dashboard).
 - Identified attacker (Attacker) → **Path B** (Deception Dashboard).
2. **User Experience:**
 - **Path A:** Fast, clean, functional, real data.
 - **Path B:** Slow, full of "convincing" errors, fake data, infinite loops.

In your specification, it is important to emphasize that both paths use the same basic CSS and HTML files (Templates), so that the deception is visually perfect. The difference lies only in the data (Data) injected into the page and the server response time.

Tech Stack Summary

Server: Node.js, Express.

DB: MongoDB, Mongoose.

Real-Time: Socket.io

Fake Data Gen: fakerLogin.js.

Geo-Location: GeoIP-lite / ip-api.

Fingerprinting: Express-UserAgent.

UI Engine: EJS, Bootstrap, Leaflet.js